STUDENT MANAGEMENT SYSTEM

1. Project Overview

The **Student Management System** is a console-based Python application developed using Object-Oriented Programming (OOP) principles. The application allows users to manage student information, update student GPA, and organize course enrollments through modular programming. It demonstrates the practical implementation of classes, inheritance, method overriding, and reusable code using multiple Python modules.

2. Problem Statement

Develop a modular Python application that manages student records and course enrollments using Object-Oriented Programming concepts. The application should create student and course objects, update student GPA, manage enrollments, demonstrate inheritance through graduate students, and organize functionality into separate Python modules.

3. Features

Student Management

- Create student records
- Display student details
- Update student GPA

Course Management

- Add students to courses
- Remove students
- Display enrolled students
- Prevent duplicate enrollment

OOP Features

- Classes and Objects
- Inheritance
- Method Overriding
- Modular Programming

4. Technologies Used

- Python 3
- Visual Studio Code
- Command Prompt / Terminal
- Modular Programming

5. Python Concepts Used

- Object-Oriented Programming (OOP)
- Classes and Objects
- Constructors (__init__)
- Inheritance
- Method Overriding
- Encapsulation
- Modular Programming
- Lists
- Conditional Statements
- Loops
- User Input Handling
- String Formatting

6. Program Workflow

Start

↓

Create Student and Course Objects

↓

Enroll Students in Course

↓

Display Student Information

↓

Update Student GPA

↓

Remove Student

↓

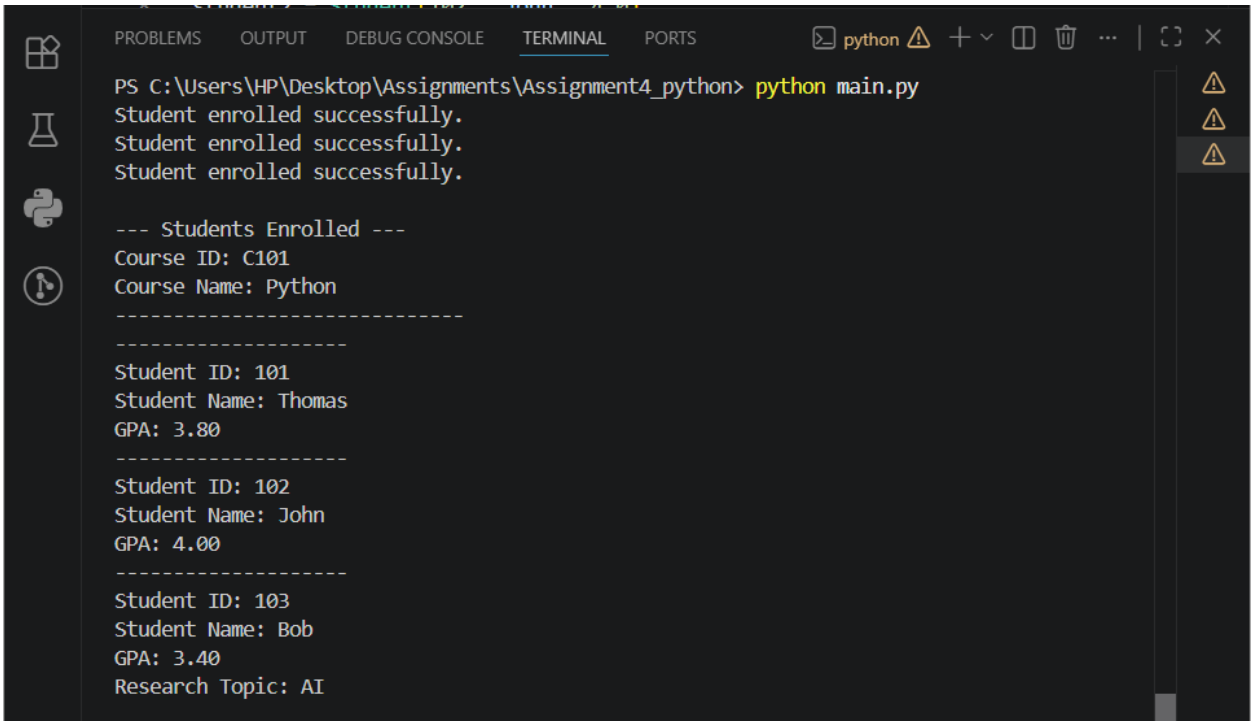
Display Updated Student List

↓

End

7. Output Screenshots and Explanation

Screenshot 1: Students Enrolled



Explanation

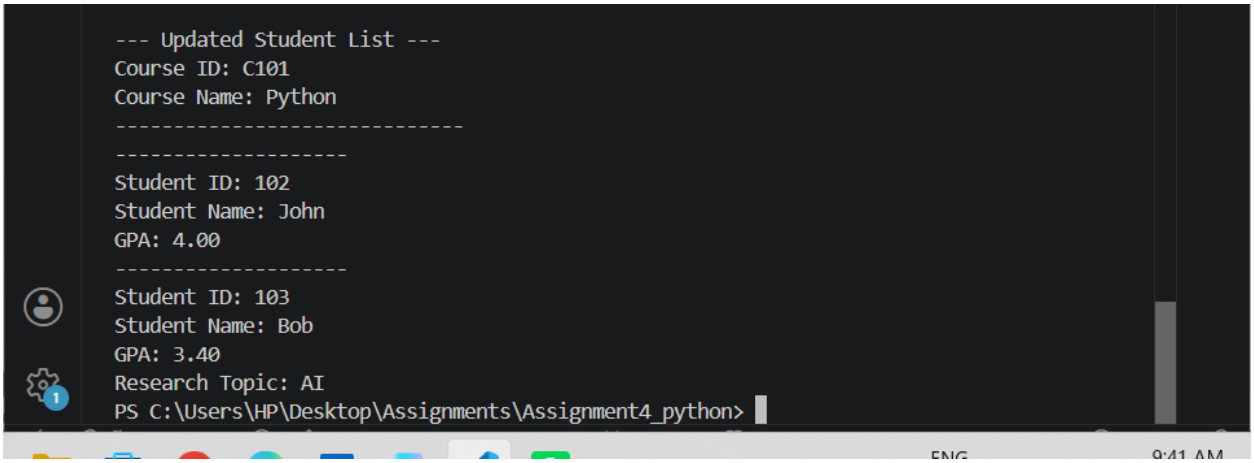
The application begins by creating two Student objects and one GraduateStudent object. These students are enrolled in the Python course using the add_student() method. The program then displays the course details along with the information of all enrolled students, including Student ID, Student Name, GPA, and the Research Topic for the graduate student. This demonstrates object creation, inheritance, and course enrollment functionality.

Screenshot 2: GPA Update and Student Removal

Explanation

The application successfully updates the student's GPA after receiving a valid input and removes the selected student from the course using a valid Student ID. The displayed messages confirm that both operations have been completed successfully.

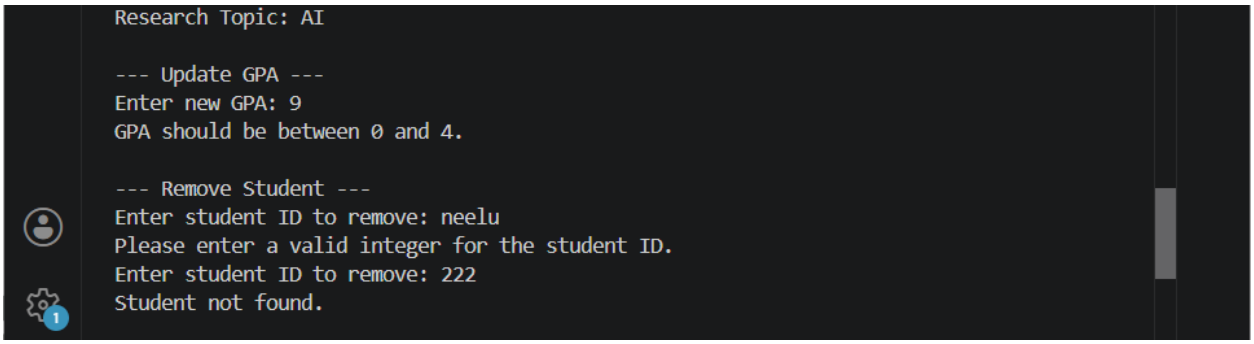
Screenshot 3: Updated Student List



Explanation

After removing the selected student, the application displays the updated list of enrolled students. The output confirms that the student with ID **101 (Thomas)** has been removed successfully, while the remaining students continue to be enrolled in the course. This verifies that the removal operation has been performed correctly.

Screenshot 4: Exception Handling



Explanation

The application demonstrates exception handling by validating user inputs during execution. When an invalid GPA value (9) is entered, the program displays the message **"GPA should be between 0 and 4."** without terminating unexpectedly. Similarly, when a non-integer value (neelu) is entered instead of a Student ID, the application catches the invalid input and prompts the user to enter a valid integer. Finally, when a non-existent Student ID (222) is entered, the application displays **"Student not found."** These validations ensure the program remains robust and user-friendly while preventing invalid operations.

8. Learning Outcomes

After completing this project, I learned how to:

- Design reusable classes and objects
- Apply inheritance and method overriding
- Organize programs using modular programming
- Manage relationships between students and courses
- Validate user input using exception handling
- Improve code maintainability through module separation
- Strengthen my understanding of Object-Oriented Programming concepts

9. Conclusion

The Student Management System successfully demonstrates the implementation of Object-Oriented Programming concepts in Python. The project applies classes, objects, constructors, inheritance, method overriding, and modular programming to build a structured and maintainable application. It also provides practical experience in designing modular Python applications and organizing code into reusable modules. Overall, this project strengthened the understanding of core OOP principles and their real-world application in software development.